

***Your container works perfectly..
but attackers may already have
everything they need inside it 🤔***

What is Container Hardening?

- Process of securing containers to reduce security risks
- Goal:
- Minimize attack surface
- Prevent unauthorized access
- Reduce exploitable vulnerabilities

Why Containers Become Vulnerable?

- Containers often include:
- Unnecessary packages
- Shell utilities
- Debugging tools
- Old libraries
- Attackers can abuse these components after compromise

What is Attack Surface?

- Total number of entry points attackers can exploit
- More software/components inside container:
- More chances of vulnerabilities
- “Smaller container = Smaller attack surface”

Common Container Security Risks

- Vulnerable base images
- Running containers as root
- Hardcoded secrets
- Unpatched packages
- Exposed ports
- Insecure dependencies

How Container Hardening Works?

- Removing unnecessary tools/packages
- Using minimal base images
- Limiting permissions
- Keeping images updated
- Scanning containers regularly

What are Distroless Images?

- Minimal container images containing:
- Only application runtime
- No package manager
- No shell
- No unnecessary OS tools
- Built specifically for security & minimalism

Why Distroless Images are Important?

Principle of Least Privilege

Container Vulnerability Scanning

Industry-Standard Container Security Tool

Typical Secure Container Workflow

- Build container image
- Scan image for vulnerabilities
- Remove unnecessary packages
- Harden container
- Deploy only secure images